

CSCE 629 ANALYSIS OF ALGORITHMS · MODULE 13 · LECTURE NOTES

Approximation Algorithms

Provably close, when exactly right is unavailable.

MODULE OUTCOMES

Define approximation ratio and distinguish it from heuristic performance.

Analyse approximation ratios without knowing the optimum.

Apply the standard techniques: greedy, LP rounding, local search.

Explain PTAS and FPTAS and give an example of each.

Explain hardness of approximation results.

1 What a guarantee is

Module 12 established that for NP-hard problems you will not get an exact polynomial algorithm. One response is to accept a solution that is provably close.

minimisation: $ALG \leq \rho \cdot OPT$ **maximisation:** $ALG \geq OPT / \rho$

APPROXIMATION VERSUS HEURISTIC

An **approximation algorithm** comes with a ratio ρ that holds on every input, proved.

A **heuristic** has no such guarantee. It may be far better than the approximation algorithm on typical data — local search and simulated annealing usually are — and it may be arbitrarily bad on some input, with no way to know in advance which.

Neither is strictly better. The question is whether you need a statement you can defend, or an answer that is usually good.

1.1 The central difficulty, and the standard manoeuvre

To prove $ALG \leq 2 \cdot OPT$ you would seem to need OPT . But if you could compute OPT you would not be approximating.

BOUND AGAINST SOMETHING COMPUTABLE

Find a quantity L you can compute, satisfying $L \leq \text{OPT}$ (for minimisation). Then show $\text{ALG} \leq \rho \cdot L$, and transitivity gives $\text{ALG} \leq \rho \cdot \text{OPT}$.

Every analysis in this module is this manoeuvre, and identifying the right L is the creative step. For vertex cover it is the size of a matching; for metric TSP it is the weight of a minimum spanning tree; for LP-based methods it is the value of the relaxation.

2 Vertex cover: a 2-approximation

```
C = set()
while there is an uncovered edge (u, v):
    C.add(u); C.add(v)          # take BOTH endpoints
    remove every edge incident to u or v
return C
```

Analysis. The edges selected by the loop form a **matching** M : after selecting (u,v) we delete every edge touching u or v , so no later selected edge shares a vertex with it.

Any vertex cover must contain at least one endpoint of every edge in M . Since the edges of M are vertex-disjoint, those endpoints are all distinct, so any cover has size at least $|M|$. In particular $\text{OPT} \geq |M|$ — the computable lower bound.

The algorithm adds exactly two vertices per edge of M , so $|C| = 2|M| \leq 2 \cdot \text{OPT}$. ■

TAKING BOTH ENDPOINTS LOOKS WASTEFUL, AND IS THE POINT

The obvious improvement — take only the higher-degree endpoint — is a better *heuristic* and has no provable constant ratio. It can be made to perform arbitrarily badly relative to the optimum.

This is a recurring pattern: the algorithm with the clean guarantee is often not the one with the best typical behaviour. Knowing which you need is the engineering judgement.

3 LP relaxation and rounding

The most general technique in the module, and the bridge to CSCE 669.

- 1 Write the problem as an **integer program**. For vertex cover: minimise $\sum x_v$ subject to $x_u + x_v \geq 1$ for every edge, with $x_v \in \{0,1\}$.
- 2 **Relax** the integrality constraint to $0 \leq x_v \leq 1$. The result is a linear program, solvable in polynomial time.
- 3 The LP optimum is a lower bound on the integer optimum, since every integer solution is also a feasible fractional one. That is your computable bound, obtained for free.
- 4 **Round** the fractional solution to an integral one and bound the loss.

For vertex cover, every edge constraint $x_u + x_v \geq 1$ forces at least one endpoint to have value $\geq 1/2$. Round every $x_v \geq 1/2$ up to 1 and the rest down to 0. The result is a valid cover, and rounding at most doubles each variable, so the cost is at most twice the LP optimum, hence at most twice OPT.

A 2-approximation again, by a completely different route. The **integrality gap** — the worst-case ratio between the integer and fractional optima — limits what any LP-rounding method can achieve, which makes it a useful thing to know about a formulation before investing in it.

4 Greedy set cover: $\ln n$, and provably optimal

Given a universe of n elements and a collection of subsets, choose the fewest subsets whose union is everything. Greedy: repeatedly take the set covering the most currently uncovered elements.

Analysis. Suppose the optimum uses OPT sets. Those OPT sets cover everything, so at any moment some set covers at least a $1/\text{OPT}$ fraction of the remaining uncovered elements — otherwise the optimal sets could not cover the remainder between them. Greedy takes a set at least that good, so after k steps:

$$\text{uncovered} \leq n(1 - 1/\text{OPT})^k < n \cdot e^{-k/\text{OPT}}$$

This drops below 1 once $k > \text{OPT} \cdot \ln n$, so greedy uses at most $\text{OPT} \cdot \ln n$ sets: an $H_n \approx \ln n$ approximation.

AND NOTHING DOES BETTER

Feige proved in 1998 that no polynomial-time algorithm achieves a ratio better than $(1 - o(1)) \ln n$ for set cover, unless $P = NP$.

So the obvious greedy algorithm is, up to lower-order terms, the best possible. This is unusual and worth noticing: normally the first algorithm you think of is not optimal, and here it provably is.

5 Metric TSP

General TSP admits no constant-factor approximation unless $P = NP$. Assume the **triangle inequality** — automatic for geometric distances — and the picture changes completely.

5.1 A 2-approximation from the MST

- 1 Build a minimum spanning tree.
- 2 Walk it depth-first, listing vertices as first encountered.
- 3 Shortcut past repeated vertices.

Analysis. Deleting any one edge from an optimal tour leaves a spanning path, which is a spanning tree, so $\text{MST} \leq \text{OPT}$ — the computable bound. A full depth-first traversal uses every tree edge twice, costing $2 \cdot \text{MST}$. Shortcutting replaces a detour by a direct edge, which the triangle inequality guarantees is no longer. So the tour costs at most $2 \cdot \text{MST} \leq 2 \cdot \text{OPT}$. ■

5.2 Christofides

Instead of doubling every tree edge, add a minimum-weight perfect matching on the odd-degree vertices of the MST. The result has all even degrees, so it has an Eulerian circuit, which shortcuts to a tour. The matching costs at most $\text{OPT}/2$, giving a **1.5-approximation**.

Published in 1976, it stood as the best known ratio for metric TSP until 2020, when it was improved by a margin of roughly 10^{-36} . That forty-four-year gap is a reasonable indication of how hard progress here is.

6 Schemes

	PTAS	FPTAS
Guarantee	$(1 + \epsilon)$ for any fixed $\epsilon > 0$	$(1 + \epsilon)$ for any $\epsilon > 0$
Running time	Polynomial in n for each fixed ϵ ; dependence on $1/\epsilon$ may be arbitrary	Polynomial in both n and $1/\epsilon$
The catch	$n^{1/\epsilon}$ is a legal PTAS. At $\epsilon = 0.1$ that is n^{10} — polynomial and entirely unusable.	None. Genuinely practical.
Example	Euclidean TSP (Arora, Mitchell)	0/1 knapsack, by rounding the values and running the DP of Module 10

6.1 Hardness of approximation

How well a problem can be approximated is itself a question with theorems attached, and the answers vary enormously:

Problem	Best known	Hardness
Knapsack	FPTAS	None — arbitrarily close, efficiently.
Vertex cover	2	No better than 1.36 unless $P = NP$; 2 may be optimal under the Unique Games Conjecture.
Metric TSP	1.5 (Christofides)	No better than 123/122.
Set cover	$\ln n$	$\ln n$ is optimal (Feige).
General TSP	None	No constant factor unless $P = NP$.
Max clique	None useful	Inapproximable within $n^{1-\epsilon}$.

Before designing an approximation algorithm, look up what is known. Discovering that your target ratio is provably unachievable saves a great deal of time, and discovering that an FPTAS exists saves even more.

7 Choosing in practice

Need	Choice
A guarantee you can state to a third party — a contract, a safety case, a published bound	Approximation algorithm.
Best typical quality, no guarantee required	Heuristic: local search, simulated annealing, or a tuned greedy.
Optimality on small instances	Exact exponential: branch and bound, or DP over subsets.

Need	Choice
Both a bound and a good answer	Run both. The approximation bounds the optimum, which tells you how much room the heuristic has left — often the most useful information available.

WHERE THIS COURSE LEAVES YOU

You can analyse an algorithm and say honestly where the analysis stops describing the machine. You can pick a technique from a problem's structure and prove the choice correct. You can see that an unfamiliar problem is really graph search, flow, or dynamic programming. And you can tell when to stop looking for an exact algorithm and what to do instead.

CSCE 614, running alongside, explains the machine this course abstracted away. **CSCE 735** repeats the exercise for parallel hardware. **CSCE 641** is where the spatial data structures and the cost model start earning their keep.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.046J — Approximation Algorithms	Vertex cover, set cover, and TSP with the ratio proofs done carefully.
Williamson & Shmoys, The Design of Approximation Algorithms (free PDF)	The standard graduate text, released free by the authors. Organised by technique, which is the right way round.
Jeff Erickson, Algorithms — Approximation Algorithms chapter	A shorter introduction with the key proofs, good as a first pass.
Vazirani, Approximation Algorithms (author's page)	The other standard reference, strong on LP duality and primal-dual methods.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement the matching-based 2-approximation for vertex cover. Compare against a brute-force exact solver on small random graphs and record the observed ratios — they will mostly be well under 2.
- 2 Implement the 'pick the highest-degree vertex' heuristic for vertex cover. Find a family of graphs on which its ratio grows without bound.

- 3 Implement greedy set cover and construct an instance where it genuinely needs about $\ln n$ times the optimum.
- 4 Implement the MST-based 2-approximation for metric TSP on random Euclidean points. Compare against an exact Held–Karp solution for $n \leq 15$, and report the observed ratios.
- 5 Implement the FPTAS for knapsack by scaling and rounding values, then running the Module 10 DP. Plot solution quality against running time as ϵ varies.
- 6 Implement a local-search heuristic for TSP (2-opt). Compare it against the MST approximation on 1,000-city instances: which gives better tours, and which gives you a bound?
- 7 **Project 2 is now due.** Assemble the three-way comparison described in the syllabus — exact, approximate with a proved ratio, and heuristic — on your chosen NP-hard problem.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define approximation ratio for minimisation and maximisation, and state what distinguishes an approximation algorithm from a heuristic.
- 2 How do you prove $\text{ALG} \leq 2 \cdot \text{OPT}$ without being able to compute OPT ?
- 3 Give the computable lower bound used in the vertex cover proof, and explain why it is a lower bound.
- 4 Why does taking both endpoints give a provable ratio while taking the higher-degree endpoint does not?
- 5 How does LP relaxation supply a lower bound for free, and what is an integrality gap?
- 6 What is the difference between a PTAS and an FPTAS, and why does it matter?
- 7 Name a problem with an FPTAS and one with no constant-factor approximation at all.